

Lattice-Propagated k-Nearest Neighbors

Devin Park
TJHSST Machine Learning 1
TJHSST
Alexandria, VA
2026dpark@tjhsst.edu

Kevin Chen
TJHSST Machine Learning 1
TJHSST
Alexandria, VA
2026kchen@tjhsst.edu

Anna Park
TJHSST Machine Learning 1
TJHSST
Alexandria, VA
2026apark@tjhsst.edu

Abstract—Supervised learning algorithms often rely on large and densely labeled datasets, which are not always available in real-world settings. One such scenario occurs during the early stages of presidential elections, when only partial results are released while accurate predictions and inferences are still desired. In these settings, traditional supervised learning algorithms can suffer from high variance and unreliable generalization due to limited labeled data.

In this paper, we propose *Lattice-Propagated k-Nearest Neighbors (LP-kNN)*, a hybrid classification algorithm that extends the neighbor-based intuition of *k-Nearest Neighbors* by propagating labels across a discretized lattice. By framing classification as an iterative lattice-filling process, LP-kNN is able to infer labels in regions of the feature space where training data is sparse or unavailable. This approach enables effective classification under limited data availability and provides a structured alternative to standard instance-based methods.

Index Terms—kNN, LP-kNN, modified KMeans, lattice, propagation

I. INTRODUCTION

Supervised learning algorithms often relies on large and densely labeled datasets. However, in many real world applications, finding a sufficiently large dataset that is readily available is a difficult problem faced by many. One such example is during the early stages of presidential-elections when only partial results are released while predictions and inferences are heavily desired. In these cases, traditional supervised learning algorithms can suffer from high variance and naive generalizations due to their dependence to datasets.

KNN is a simple yet effective classification algorithm whose predictions rely on the closeness to other neighboring data points. Despite its simplicity and flexibility, KNN scales poorly with datasets size and sparsely labeled data. Each computations requires computing the distance between instances and its quality degrades when there are insufficient labeled samples. These limitations lead us to consider alternatives that can take advantage of structural assumptions about our data to propagate information across various locations.

In this paper, we propose *Lattice-Propagated k-Nearest Neighbors (LP-kNN)*, a hybrid classification algorithm that combines the philosophy of kNN’s neighbor based predictions as the corner stone for propagation to label up a discretized lattice. By transforming the classification as a lattice filling process, our algorithm can infer labels in regions where data was unprovided.

The main contributions of this paper are as follows:

- C1. We introduce LP-kNN, a novel lattice-based extension of KNN that propagates labels through discretized feature space.
- C2. We demonstrate how lattice propagation enables effective classification in scenarios with sparse labeled data.
- C3. We empirically evaluate LP-kNN against traditional KNN and modified KMeans on a real-world election dataset.
- C4. We provide visual analysis of the propagation process to illustrate the algorithm’s behavior and convergence.

II. RELATED WORKS

All of references [1], [2], and [4] all discuss new variants of KNN in order to reduce the computational cost while maintaining or improving its accuracy. References [1] and [2] used a fixed radius of neighbors. Reference [4] used the majority class of the k non-nearest samples. This is similar to our approach as we adopt the fixed radius idea due to our intuition on how political ideology can only spread so far in the real world. They differ because they still use the training points for classification rather than using a lattice structure that predicts the class label for every fixed point in a lattice structure like a coordinate plane. These approaches were pretty clever because they avoid the distance calculation for every data point and only using select data points to find a trend.

Reference [5] uses the concept of cellular automata, the concept where cells on a grid change their state based on rules applied to its neighbors, to create a Multiple Classifier System for decision making. Their strengths were problem independency, robustness to noise, the lack of need of user input. A weakness of this method is it involves more than the other simple classifiers. We thought this approach was very unique due to the niche concept as well as the similarities to Random Forest. We used the idea of cellular automata with addition to KNN in order to create a lattice where each lattice point is defined by the majority class of its neighbors.

Similarly, reference [3] uses graphs to encode each data point, where the edge weights encode similarity between data points. Learning is done by propagating label information across the graph to classify new points. Their model is robust to noise, but not really scalable as computational requirements grow as well. Its similar to our work as we also take this idea of where each lattice depends on the ones around it.

III. DATASET AND FEATURES

Our dataset is the U.S. General Elections 2018 - Analysis Dataset from the MIT Election Data + Science Lab[6]. Initially, this dataset contains 3114 instances with 39 features. For preprocessing, we kept the state name, county name, votes for Donald Trump in 2016, and votes for Hillary Clinton in 2016. Using the state and county names, we used Nominatim, an open-source geocoding API with OpenStreetMap data, to obtain the latitude and longitude for each county. There were 4 outliers so we removed them. We also created our new class variable, alignment. Alignment is based on how each county voted in the 2016 presidential election. If a majority of the county voted for Donald Trump, then that county would be labeled as Republican. If a majority of the county voted for Hillary Clinton, then the county would be labeled Democrat. After preprocessing, the dataset has 3114 instances with 3 features. An example of the dataset after processing can be seen in Table I. To replicate early stages of the

TABLE I
EXAMPLES OF PREPROCESSED INSTANCES

Latitude	Longitude	Alignment
32.532237	-86.64644	republican
30.659218	-87.746067	republican
31.870253	-85.405104	republican

presidential election, we used stratified sampling to use only 20% of our preprocessed data set, making our dataset have 622 instances with 3 features. Lastly, we made sure to scale up longitude and latitude by a factor of 3 so that our model be able to make space for more lattice points in $\mathbb{Z}^2$. Afterwards, we made sure to

IV. METHODS

This section includes details on all of the Machine Learning Algorithms used for this paper.

A. K Nearest Neighbors (KNN)

We will be comparing our algorithm against the related algorithm K-Nearest Neighbors (KNN). KNN is a supervised learning algorithm that can be used for classification or regression. It is known as a "lazy learner", meaning it doesn't build a model during training. To classify a new point x , KNN finds the distance of x to every other point, then chooses the k closest points in the training set using Euclidean distance:

$$d(x, x_i) = \sqrt{\sum_{j=1}^m (x_j - x_{i,j})^2}$$

As such, this algorithm is computationally expensive due to fact it utilizes an entire dataset in order to label a single point. In classification, the predicted label is the majority class among these neighbors.

B. Modified KMeans

Another algorithm we will be comparing our algorithm against is a modified version of KMeans. KMeans Clustering is an unsupervised learning algorithm that partitions a dataset into k clusters based on feature similarity. The algorithm works by iteratively assigning each data point to the nearest cluster center, and then updating each cluster center to be the mean of all points assigned to it. Mathematically, the objective of KMeans is to minimize the within-cluster sum of squares (WCSS), also called inertia:

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

where C_j is the set of points assigned to cluster j and $\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$ is the cluster centroid. The algorithm alternates between two steps until convergence: the assignment step, where each point is assigned to the nearest centroid, and the update step, where centroids are recalculated as the mean of their assigned points.

Although KMeans is inherently unsupervised, it can be modified when labels are available. After fitting KMeans on the training data, each cluster is mapped to the majority class label of the points assigned to that cluster. New test points are then classified by assigning them to the nearest cluster center and predicting the cluster's majority label. This approach enables evaluation using standard supervised metrics such as accuracy, precision, recall, and F1 score.

C. Lattice-Propagated k-Nearest Neighbors (LP-kNN)

In this project, we introduce Lattice-Propagated k-Nearest Neighbors (LP-kNN). This model combines strategies from different machine learning algorithms to incrementally fill a lattice through the kNN heuristic. This increment, or propagation, is similar to that of cellular automata in its approach of using a simple set of rules to recursively compute the next generation.

a) *Lattice Construction:* Given our training dataset composed of two-dimensional, continuous features in $\mathbb{R}^2$, we first map the features of our instances to a finite grid by mapping them to the closest lattice point in $\mathbb{Z}^2$. We do this by rounding each of the features. Formally, given instance i with feature values as $\langle a_i^{(1)}, a_i^{(2)} \rangle \in \mathbb{R}^2$:

$$\langle a_i^{(1)}, a_i^{(2)} \rangle \rightarrow \langle \lfloor a_i^{(1)} \rfloor, \lfloor a_i^{(2)} \rfloor \rangle \in \mathbb{Z}^2$$

This mapping is demonstrated in Fig 1. After we map each feature value, we give the mapped lattice points a label. If multiple instances map to the same lattice point, the lattice point is given a label through majority voting.

Now, we calculate the min and max of each feature to create a finite lattice with:

$$\mathcal{L} = \{ (x, y) \in \mathbb{Z}^2 \mid \min_i \lfloor a_i^{(1)} \rfloor \leq x \leq \max_i \lfloor a_i^{(1)} \rfloor, \\ \min_i \lfloor a_i^{(2)} \rfloor \leq y \leq \max_i \lfloor a_i^{(2)} \rfloor \}$$

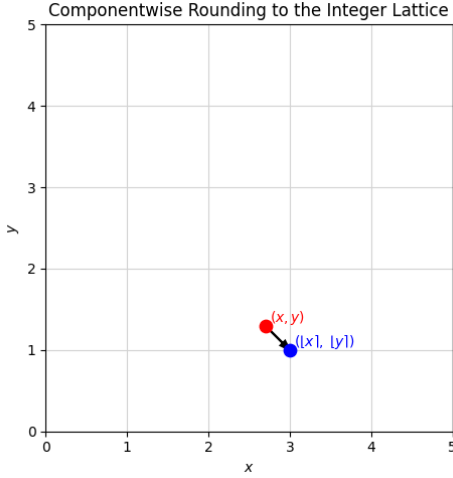


Fig. 1. Rounding points from $\mathbb{R}^2$ to $\mathbb{Z}^2$

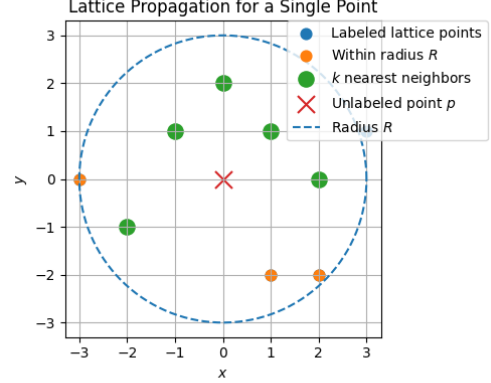


Fig. 2. Lattice Propagation Procedure Example

b) Lattice Propagation: Even after following the steps for lattice construction, not every point in our finite lattice will have an assigned label. Hence, we procedurally fill up the lattice by assigning the value based on neighboring lattice point values.

- 1) **Identify labeled and unlabeled points.** Partition the lattice into a set of labeled points $\mathcal{F} \subset \mathcal{L}$ and unlabeled points $\mathcal{U} = \mathcal{L} \setminus \mathcal{F}$.
- 2) **Select neighboring points.** For each unlabeled lattice point $p \in \mathcal{U}$, collect the set of labeled lattice points within radius R :

$$\mathcal{N}(p) = \{q \in \mathcal{F} \mid \|p - q\|_2^2 \leq R^2\}.$$

- 3) **Assign labels via k -NN.** If $|\mathcal{N}(p)| \geq k$, assign p a label using a k -nearest neighbors majority vote over $\mathcal{N}(p)$. Otherwise, defer labeling p to a later iteration.
- 4) **Update the lattice.** All newly labeled points are added to $\mathcal{F}$ simultaneously, and removed from $\mathcal{U}$.
- 5) **Iterate until completion.** Repeat this process until all lattice points are labeled, i.e., $\mathcal{U} = \emptyset$.

Fig 2 demonstrates the Lattice Propagation Procedure for filling non-labeled lattices in a graphical manner.

V. EXPERIMENTS/RESULTS/DISCUSSION

A. KNN Results

TABLE II
CONFUSION MATRIX

	Pred True	Pred False
Actual True	TP	FN
Actual False	FP	TN

In table 2, the values in the confusion matrix are labeled with values TP (true positive), FN (false negative), FP (false positive), and TN (true negative).

Accuracy is a measure that describes how many of the choices the model predicted were correct and is measured as follows:

$$\text{accuracy} = \frac{TP + TN}{TN + TP + FN + FP}$$

Precision measures the quality of a models positive predictions and is measured as such:

$$\text{precision} = \frac{TP}{TP + FP}$$

Recall measures how many of the true positive values the model identify as positive and its measured as such:

$$\text{recall} = \frac{TP}{TP + FN}$$

F1 score measures the balance between precision and recall and is measured as such:

$$\text{F1-score} = \frac{2}{\frac{1}{\text{recall}} + \frac{1}{\text{precision}}}$$

For KNN, we conducted trials where $k \in \{0, 20\}$ and recorded a confusion matrix for each trial.

We chose $k = 10$ because it had the highest accuracy. For the Democrat class, the model struggles to identify them as shown from its low recall score. When it does predict a democrat, it is only correct about 61% of the time. Conversely, the KNN model is very strong in predicting Republicans as shown from its class accuracy, recall, and precision. Not only that, its F1-score is also extremely high, demonstrating high recall and precision balance. This suggests that the model is biased towards predicting Republicans and struggles with minority-class detection which is due to class imbalance as well as possibly smoothing at higher levels of k .

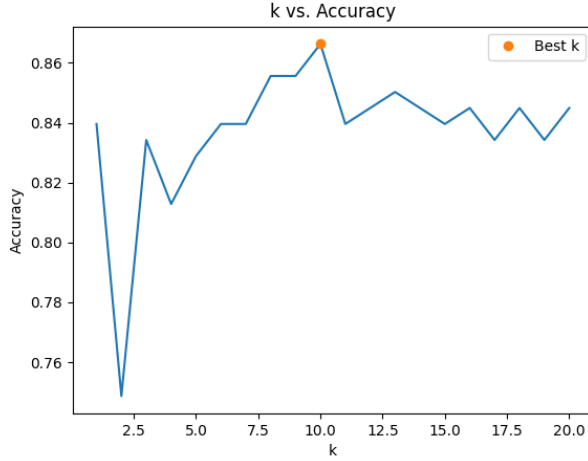


Fig. 3. Graph of k vs. accuracy

TABLE III
CONFUSION MATRIX AND CLASSIFICATION METRICS FOR KNN ($k = 10$)

Predicted Class	Actual Class		Metrics		
	Democrat	Republican	Precision	Recall	F1
Democrat	11	7	0.611	0.379	0.468
Republican	18	151	0.893	0.956	0.924
Overall Accuracy			0.866		

B. Modified KMeans Results

For KMeans, we chose $k = 11$ using the Elbow method, as seen in Fig 4. The Elbow method is a way to find the optimal number of clusters by plotting the WCSS (See Section IV-B) against the number of clusters (k) and identifying the "elbow" point.

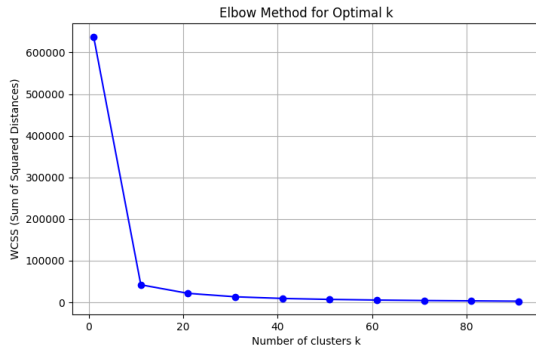


Fig. 4. Elbow Method Graph

The confusion matrix in Table IV shows that KMeans correctly classified most Republican instances, but struggled with Democrat instances.

This makes sense, because KMeans captures clusters based on geometric similarity, not labels, so it naturally performs better on the majority class (Republicans). The low recall and F1 for Democrats indicate that the clusters do not separate

minority classes well, a known limitation of unsupervised clustering for imbalanced datasets. The Elbow method helped select a reasonable k in terms of cluster compactness, but the label alignment shows that KMeans alone may not be sufficient for classification.

TABLE IV
CONFUSION MATRIX AND CLASSIFICATION METRICS FOR K-MEANS ($k = 11$)

Predicted Class	Actual Class		Metrics		
	Democrat	Republican	Precision	Recall	F1
Democrat	1	1	0.500	0.034	0.064
Republican	28	157	0.848	0.993	0.915
Overall Accuracy			0.844		

C. LP-kNN Results

In LP-kNN, we have two variables that we can change: the propagation radius R and k for nearest neighbors. For the sake of testing, our values range from $k, R \in \{0, 1, \dots, 10\}$. From

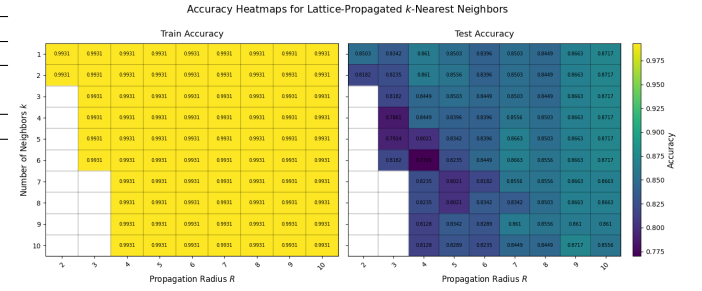


Fig. 5. Training and Test Accuracy of LP-kNN in a heatmap of $k, R \in \{0, 1, \dots, 10\}$

Fig 5, there were 7 combinations of different k and R values that gave the highest accuracy of 87.17%. As a representative, we will choose model $k = 5, R = 10$ to analyze. For the case

TABLE V
CONFUSION MATRIX AND CLASSIFICATION METRICS FOR LP-kNN ($k = 5, R = 10$).

Predicted Class	Actual Class		Metrics		
	Democrat	Republican	Precision	Recall	F1
Democrat	6	1	0.857	0.207	0.333
Republican	23	157	0.872	0.994	0.929
Overall Accuracy			0.872		

of $k = 5, R = 10$, there were 5 iterations taken before the entire lattice was filled. These five iterations were plotted and placed in the appendix Fig 6 for visual purposes where as our analysis appears in Table V.

VI. CONCLUSION/FUTURE WORK

In this paper, we evaluated several learning algorithms for predicting county-level political alignment using limited labeled data, including traditional k -Nearest Neighbors (KNN), a modified KMeans approach, and our proposed Lattice-Propagated k -Nearest Neighbors (LP-kNN). LP-kNN achieved

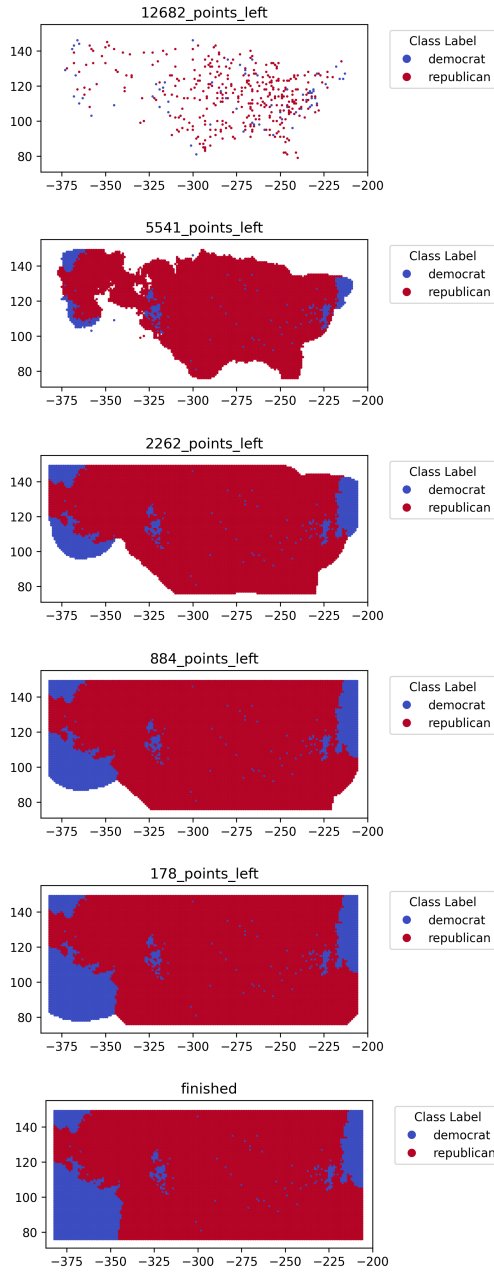


Fig. 6. Visualization of the lattice label propagation process for LP- k NN. Figures are ordered by decreasing number of unlabeled lattice points, illustrating the iterative filling procedure until convergence.

the highest overall accuracy (87.2%). The only other competitive model was KNN, which achieved higher recall and F1 scores on the Democrat (minority) class, however LP- k NN performed better on the Republican class and better overall. Therefore, we claim that our model LP- k NN was the highest performing model of the three.

The observed performance differences can be attributed to how each algorithm handles spatial structure and class imbalance. Standard KNN relies solely on local point density and is therefore sensitive to uneven class distributions, causing

it to favor the majority Republican class. Modified KMeans prioritizes geometric compactness rather than class separability, which limits its ability to distinguish minority-class regions when clusters overlap or are imbalanced. In contrast, LP- k NN explicitly models spatial continuity by propagating labels through a structured lattice, allowing information from sparsely labeled but geographically coherent regions to influence nearby unlabeled areas. This combination of local neighbor voting and global spatial structure enables LP- k NN to better capture underlying geographic patterns, resulting in improved performance on minority-class detection and a more balanced overall classification.

For future work, we could explore a wider range of propagation radii R and k to better understand their impact on classification performance. We could also investigate options for different boundaries other than a rectangle, such as circular boundaries or boundaries constrained to the geographic shape of the United States. Additionally, incorporating weighted k -NN schemes would assign greater influence to closer or more confident neighbors and could improve results. Finally, extending LP- k NN to higher-dimensional data would further test its scalability and applicability beyond purely geographic data.

VII. CONTRIBUTIONS

Devin Park worked on the LP- k NN algorithm, as well as sections I, IV, and V of the paper.

Kevin Chen worked on the preprocessing, KNN algorithm, as well as sections II, III, IV, and V of the paper.

Anna Park worked on the modified K-Means algorithm, as well as sections IV, V, and VI of the paper.

REFERENCES

- [1] P. Müller, "Flexible K Nearest Neighbors Classifier: Derivation and Application for Ion-mobility Spectrometry-based Indoor Localization," 2023 13th International Conference on Indoor Positioning and Indoor Navigation (IPIN), Nuremberg, Germany, 2023, pp. 1-6
- [2] Cano Urrego, B., Alsup, A., Thompson, J.A. et al. A stochastic approach to k-nearest neighbors search using a fixed radius method. *Comput Stat* 41, 27 (2026).
- [3] Z. Song, X. Yang, Z. Xu and I. King, "Graph-Based Semi-Supervised Learning: A Comprehensive Review," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 8174-8194, Nov. 2023
- [4] A new locally adaptive k-nearest neighbor algorithm based on discrimination class, *Knowledge-Based Systems*, Volume 204, 2020
- [5] Povalej, P., Kokol, P., Družovec, T.W., Stiglic, B. (2005). Machine-Learning with Cellular Automata. In: Famili, A.F., Kok, J.N., Peña, J.M., Siebes, A., Feelders, A. (eds) *Advances in Intelligent Data Analysis VI*. IDA 2005. Lecture Notes in Computer Science, vol 3646. Springer, Berlin, Heidelberg.
- [6] Medsl."2018-Elections-Unofficial/Election-Context-2018.csv at Master MEDSL/2018-Elections-UNOFFICIAL."GitHub, github.com/MEDSL/2018-elections-unofficial/blob/master/election-context-2018.csv.